

Reviewer Pack — Minimal Order of Quandle Structures and SAT Clause Complexit...

Entry 995817fe48d0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 14:41:42 UTC

Minimal Order of Quandle Structures and SAT Clause Complexity

Entry ID: 995817fe48d0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 14:41:42 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quandles) **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

For every Boolean satisfiability instance with n variables, the minimal order of the quandle structure associated with its clause set is $O(n^{3/2})$ and this invariant correlates with the clause complexity.

Rationale (proposer's reasoning):

Quandles provide a general algebraic framework for studying permutations and their symmetries. The minimal order of a quandle can reveal underlying combinatorial structures that influence the complexity of satisfiability problems, potentially uncovering new complexity lower bounds.

Taxonomy category: Quandles × Boolean Satisfiability (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 91eeefec28f67d14

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the calculated correlation coefficient between the minimal order of the quandle and clause complexity for all generated SAT instances with $n \leq 40$ is greater than or equal to 0.7, and no instance has a quandle order exceeding $O(n^{3/2})$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('quandle structures' AND 'Boolean satisfiability') - minimal order ('algebraic structure' OR 'quandle') AND clause complexity - (variable-free quandle) AND (SAT clause complexity)

Top relevant hits considered: - [http://arxiv.org/abs/2012.15478v3] N-quandles of links - [http://arxiv.org/abs/0905.3374v3] Symmetric Extensions of Dihedral Quandles and Triple Points of Non-orientable Surfaces - [http://arxiv.org/abs/1808.06276v2] On the knot quandle of the twist-spun trefoil - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/1607.00443v1] Algebraic Proof Complexity: Progress, Frontiers and Challenges - [http://arxiv.org/abs/1708.06805v3] Scale-Free Random SAT Instances - [http://arxiv.org/abs/1012.2923v3] Quandle homology and complex volume

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math

def generate_sat_instance(n):
    clauses = []
    for _ in range(2 * n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        clauses.append(clause)
    return clauses

def construct_quandle(clauses):
    quandle = {}
    for clause in clauses:
        key = tuple(sorted([abs(x) for x in clause]))
        if key not in quandle:
            quandle[key] = set()
        for i, x in enumerate(clause):
            for j, y in enumerate(clause):
                if i != j and (x > 0 and y < 0 or x < 0 and y > 0):
                    quandle[key].add((abs(x), abs(y)))
    return quandle

def order_of_quandle(quandle):
    visited = set()
    def dfs(node, path):
```

```

        if node in visited:
            return len(path)
        visited.add(node)
        max_length = 1
        for neighbor in quandle[node]:
            max_length = max(max_length, dfs(neighbor, path + [node]))
        visited.remove(node)
        return max_length
    order = 0
    for node in quandle:
        if node not in visited:
            order += dfs(node, [])
    return order

def clause_complexity(clauses):
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    quandle_orders = []
    clause_complexities = []

    for n in n_values:
        clauses = generate_sat_instance(n)
        quandle = construct_quandle(clauses)
        quandle_order = order_of_quandle(quandle)
        clause_complexity_val = clause_complexity(clauses)

        quandle_orders.append(quandle_order)
        clause_complexities.append(clause_complexity_val)

    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(quandle_orders, clause_complexities)) / (len(quandle_orders) - 1)
    mean_quandle_order = sum(quandle_orders) / len(quandle_orders)

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.7 and all(quandle_order <= n ** (3/2) for n in n_values),
        "counterexample": "" if correlation_coefficient >= 0.7 and all(quandle_order <= n ** (3/2) for n in n_values) else "found"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1000, 9999) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_too_low\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's conditions. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13463
2	preregistration	ollama_remote	glm4:latest	0	0	15531
3	novelty	ollama_remote	glm4:latest	0	0	8323
4	novelty	ollama_remote	glm4:latest	0	0	10064
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22576
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11163
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17722
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11716
9	judge	ollama_remote	glm4:latest	0	0	18981

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129540 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in <research/audit/995817fe48d0.jsonl>)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/995817fe48d0.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/995817fe48d0.tar.gz` (if generated)